



Dokumentacja oprogramowania

Spis treści:

- 2. O programie, możliwości, źródła
- 3. Dokumentacja wstępna – ogólne wymagania
- 4. Opisanie głównych funkcji oprogramowania – ustawienia, tablica wyników, „credits”, „Game Selector”, „Game Lobby” – przyłączanie się do gry
- 5 - 11. Opisanie głównych funkcji oprogramowania – pokój graczy, rozgrywka, bot
- 12 - 13. Projekt i realizacja systemu
- 14. Protokół rozbieżności, autorzy

THE SHIP GAME



O programie

Oprogramowanie "THE SHIP GAME" to gra napisana w języku C# .NET, wykorzystująca bibliotekę Microsoft Forms do interfejsu graficznego. Jest to cyfrowa wersja gry planszowej opartej na tematyce gry w statki.

Możliwości

- Rozgrywka pomiędzy dwoma graczami podłączonymi do tej samej sieci
- Rozgrywka przeciwko komputerowi
- Ustawienie motywu
- Odtwarzanie muzyki
- Ustawienia głośności
- Ustawienia pseudonimu
- Ustawienia motywu
- Ustawienia rozdzielności
- Statystyki
- Proste animacje
- Przyjazny interfejs w stylu retro
- Auto pozycjonowanie obiektów planszy

Zewnętrzne źródła

- Plik muzyczny – Boss Time www.FesliyanStudios.com
- Czcionka - www.cufonfonts.com/font/super-mario-ds



Dokumentacja THE SHIP GAME – wstępna (ogólne wymagania)

Postanowienia klienta Wymogiem klienta jest utworzenie cyfrowej wersji planszowej gry o tematyce gry w statki. Oprogramowanie ma być napisane w języku C# .NET z wykorzystaniem Microsoft Forms do interfejsu graficznego.

Postanowienia zespołu Oprogramowanie będzie wzbogacone w szatę graficzną w stylu retro. Gra będzie umożliwiała rozgrywkę na dwóch komputerach przyłączonych do tej samej sieci LAN, komunikacja między nimi będzie odbywać się asynchronicznie, aby nie blokować interfejsu. Dodatkowo oprogramowanie będzie umożliwiała rozgrywkę przeciwko komputerowi. Program będzie zbierał statystyki tj. liczba wygranych oraz liczba trafionych części planszy. Dodatkowo odtwarzana będzie melodia w tle w celu umilenia rozgrywki. Oprogramowanie zostanie wzbogacone w kilka kontrolerek, przy czym każda z nich będzie unikatowa. Zostanie utworzony system komunikatów, pasujący do przyjętego interfejsu graficznego.

Język docelowy: angielski
Czcionka: MarioDS



1) Opisanie głównych funkcji oprogramowania

- **Ustawienia**
Ustawienia mają umożliwiać zmianę głośności odtwarzania melodii, pseudonimu, motywu oraz rozdzielczości. Kontrolka odpowiada za odtwarzanie melodii oraz wczytywanie i zapisywanie ustawień do pliku tekstowego. W przypadku jego braku wczytywane są wartości domyślne oraz tworzony jest nowy plik konfiguracyjny.
- **Tablica wyników**
Tablica wyników umożliwia odczytanie plików, których zawartością są: liczba wygranych rozgrywek, liczba rozegranych gier, liczba zniszczonych części podczas rozgrywki.
- **Kontrolka „Credits”**
Przedstawia autorów oprogramowania.
- **Kontrolka „Game Selector”**
Odpowiada za menu wyboru rozgrywki. Umożliwia wybranie trzech pozycji. Pierwszą z nich jest „PLAY AS HOST”. Ten tryb odpowiada za dystrybucję rozgrywki. Użytkownik staje się pewnego rodzaju hostem, a jego urządzenie serwerem udostępniającym slot drugiemu użytkownikowi. „PLAY WITH FRIEND” jest podobne w funkcjonalności do „PLAY AS HOST”, aczkolwiek rozgrywka system ma miejsce od drugiej strony. Tym razem użytkownik, który wybrał ten tryb jest pewnego rodzaju klientem, który przyłącza się do rozgrywki do drugiego gracza – hosta. Ostatnim wyborem jest „PLAY WITH BOT”, ten tryb umożliwia rozgrywkę przeciwko komputerowi.
- **Kontrolka „GAME LOBBY” - > Przyłączanie się do gry**
Gdy w menu „Game Selector” klikniemy przycisk „PLAY WITH FRIEND” oczom użytkownika ukaże się interfejs, który jest konieczny do połączenia się z drugim graczem – hostem. W pole adresu użytkownik będzie musiał wpisać adres ip oraz port udostępniany przez drugiego użytkownika. Pole wprowadzania danych, jest odporne na błędy użytkownika i pomoże mu wpisać poprawny adres. Dodatkowo, „pod kontrolka” odpowiada za sprawdzenie połączenia z drugim graczem oraz wysła najpotrzebniejsze dane do ustalenia połączenia.
- **Kontrolka „GAME LOBBY” - > Lobby**
Przedstawiona „pod kontrolka” umożliwia szereg możliwości. Wyświetla pseudonimy graczy oraz ich statusy gotowości. Na dole ukazany jest adres oraz port serwera, do którego przyłączy/przyłączył się drugi rozgrywający. Głównym zadaniem jest umożliwienie użytkownikowi ustawienie pozycji „statków”. Ponadto kontrolka została

wzbogacona o metodę, która umożliwia automatyczne upozycjonowanie elementów. Kontrolka odpowiada za interpretację oraz odbiór gotowości drugiego użytkownika oraz odbiór/wysyłanie planszy.



- Kontrolka „GAME LOBBY” -> Gra
Gdy obaj gracze będą gotowi oraz ich plansze będą przygotowane to gra przełączy się w status rozgrywki. Na początku losowany jest gracz, który rozpoczyna rozgrywkę. Wtedy ten oddaje strzał i po stronie hosta sprawdzany jest rezultat trafienia. Gdy taki nastąpi lub też nie, drugiemu graczowi zostanie przedstawiony rezultat strzału oraz jego koordynaty. Wraz z tym zmieni się pole planszy oraz nastąpi zamiana kolejności. Dodatkowo kontrolka odpowiada za sprawdzanie, czy któryś z graczy nie wygrał. Gdy gra się zakończy to wyskoczy komunikat o wygranej/przegranej oraz wyskoczy propozycja rozegrania gry ponownie.
- Kontrolka „Bot Game”
Posiada większość funkcji, które są wykorzystywane w kontrolce „Bot” oraz klasie „AI”. Zaimplementowany do programu został skrypt który wstępnie generuje tablicę bota.

```
bool isInBounds(int i)
{
    return i < 10 && i >= 0;
}

return matrixx;

bool placeships(int line, int row)
{
    if (shipOrientation == ShipOrientation.Vertical)
    {
        for (int i = 0; i < currentShipSize; ++i)
        {
            // najpierw sprawdz czy nie wyjdzie poza boundsy, potem czy nie koliduje z innym statkiem
            int l = line + i;

            if (!isInBounds(l) || shipCords[l, row])
            {
                return false;
            }

            // sprawdź czy na polach dookoła (ściany dłuższe) nie ma statku

            int row_to_check = row + 1;
            if (isInBounds(row_to_check) && shipCords[l, row_to_check]) // sprawdza pola na prawo od statku
            {
                return false;
            }

            row_to_check = row - 1;
            if (isInBounds(row_to_check) && shipCords[l, row_to_check]) // sprawdza pola na lewo od statku
            {
                return false;
            }
        }
    }
}
```

Skrypt sprawdza czy statek który miałby być rozstawiony mieści się w naszej tabeli za pomocą metody (isInBounds) po czym sprawdza odpowiednio pola na prawo, na lewo, w górę i w dół czy w miejscu gdzie ma zostać postawiony statek nie znajduje się już jakiś inny obiekt.

To czy statek będzie postawiony Wertykalnie czy Horyzontalnie zależy od wylosowanej liczby (1 lub 0) przez generator liczb.

THE SHIP GAME



```
int shipsPlaced = 0;
// Console.WriteLine("hello\n");
shipOrientation = ShipOrientation.Vertical;
currentShipSize = 4;
while (true)
{
    if (GenerateRandomNumber(0, 1) == 0)
    {
        shipOrientation = ShipOrientation.Horizontal;
    }
    else
    {
        shipOrientation = ShipOrientation.Vertical;
    }
    if (Placeships(GenerateRandomNumber(0, 9), GenerateRandomNumber(0, 9)))
    {
        shipsPlaced++;
    }

    if (shipsPlaced == 1)
    {
        currentShipSize = 3;
        Bot_GameLobbyView.ships_size_three--;
        Bot_Game_plan.ships_size_three++;
    }

    if (shipsPlaced == 3)
    {
        currentShipSize = 2;
        Bot_GameLobbyView.ships_size_two--;
        Bot_Game_plan.ships_size_two++;
    }

    if (shipsPlaced == 7)
    {
        currentShipSize = 1;
        Bot_GameLobbyView.ships_size_one--;
        Bot_Game_plan.ships_size_one++;
    }

    if (shipsPlaced == 11) break;
}
```

Rozstawianie statków zaczynamy od największego rozmiaru (4) i kończymy na czterech statkach o wielkości 1.

Całość wywoływana jest już w kontrolce “Bot” metodzie “setupGame” którą wywołujemy w momencie kiedy klikamy guzik ready. Wtedy zostaje wylosowane która strona zaczyna grę, Gracz czy bot.

```
void setupGame()
{
    czasGry.Start();
    czasGry.Reset();

    AI ai = new AI();
    ClientArray = ai.GenerateMatrix();
    string m = "";
    for (int i = 0; i < 10; i++)
    {
        for (int j = 0; j < 10; j++)
        {
            if (ClientArray[i, j])
            {
                m += "# ";
            }
            else
            {
                m += "- ";
            }
        }
        m += "\n";
    }

    //MessageBox.Show("botArray\n" + m);
    for (int i = 0; i < 10; ++i)
    {
        for (int j = 0; j < 10; ++j)
        {
            var picb = tableLayoutPanel12.GetControlFromPosition(j + 1, i + 1) as PictureBox;
            //if (images[i * 10 + j] != null) {
            picb.BackgroundImage = images[i * 10 + j];
            //}
        }
    }
}
```

THE SHIP GAME



```
private void CheckIfBothReady()
{
    // Random random = new Random();
    // int x_bot = random.Next(0, 9);
    // int y_bot = random.Next(0, 9);

    generate();
    Random rnd = new Random();
    int index = rnd.Next(lista.Count);
    string pozycja = (string)lista[index];
    lista.RemoveAt(index);
    int x_bot = int.Parse(pozycja.Split(';')[0]);
    int y_bot = int.Parse(pozycja.Split(';')[1]);

    SwitchTab(1);
    setupGame();

    HostArray = shipCords;

    bool ktozaczyna = extras.Draw();
    if (ktozaczyna)
    {
        ItIsMyTurn = true;

        //ja
    }
    else
    {
        //MessageBox.Show("Bot zaczyna" + "strzela w " + y_bot.ToString() + " " + x_bot.ToString());
        // Control cellControll = tableLayoutPanel12.GetControlFromPosition(x_bot, y_bot);
        string m = "";
        for (int i = 0; i < 10; i++)
        {
            for (int j = 0; j < 10; j++)

                else
                {
                    //MessageBox.Show("Bot zaczyna" + "strzela w " + y_bot.ToString() + " " + x_bot.ToString());
                    // Control cellControll = tableLayoutPanel12.GetControlFromPosition(x_bot, y_bot);
                    string m = "";
                    for (int i = 0; i < 10; i++)
                    {
                        for (int j = 0; j < 10; j++)
                        {
                            if (HostArray[i, j])
                            {
                                m += "# ";
                            }
                            else
                            {
                                m += "- ";
                            }
                        }
                        m += "\n";
                    }
                    //MessageBox.Show("moj arrr\n" + m);

                    if (HostArray[x_bot, y_bot] == true)
                    {
                        AI.Bot_Game_plan.total_amount_shots++;

                        AI.Bot_Game_plan.amount_bot_good_shots++;
                        AI.Bot_Game_plan.bot_good_shots_position[x_bot, y_bot] = true;
                        AI.Bot_Game_plan.bot_moves[x_bot, y_bot] = true;
                        //MessageBox.Show("trafites");
                        AI.Bot_Game_plan.shooted = true;
                        AI.Bot_Game_plan.last_good_shot_x = x_bot;
                        AI.Bot_Game_plan.last_good_shot_y = y_bot;
                        getMyPictureBox(x_bot, y_bot).BackgroundImage = imgX;
                        AI.Bot_Game_plan.total_amount_ships--;
                    }
                    else
                    {
                        AI.Bot_Game_plan.total_amount_shots++;
                        AI.Bot_Game_plan.bot_moves[x_bot, y_bot] = true;
                        getMyPictureBox(x_bot, y_bot).BackgroundImage = imgMissed;
                    }
                }
            }
        }
    }
}
```

THE SHIP GAME



Wszystkie możliwe ruchy bota są zapisane w liście. Zostaje wygenerowana liczba od 0 do długości listy i wylosowana pozycja to aktualny ruch bota (w momencie wylosowania ruchu pozycja z listy zostaje usunięta oraz zakres zmniejszony o 1. Pozycje są przechowywane w postaci stringa a następnie splitowane, aby móc przechowywać pozycje x oraz y.

```
Random random = new Random();
//int x_bot = random.Next(0, 9);
// int y_bot = random.Next(0, 9);
int index = random.Next(0, lista.Count);
string pozycja = (string)lista[index];

int x_bot = int.Parse(pozycja.Split(';')[0]);
int y_bot = int.Parse(pozycja.Split(';')[1]);
```

```
private void generate()
{
    for (int i = 0; i < 10; i++)
    {
        for (int j = 0; j < 10; j++)
        {
            lista.Add(i.ToString() + ';' + j.ToString());
        }
    }
}
```

Odwrot: 0

Do momentu kiedy bot nie odda pierwszego poprawnego strzału działa całkowicie pseudolosowo. W momencie kiedy zostanie ustrzelona pierwsza pozycja, bot przechodzi do kolejnych warunków które skanują poszczególne pola, przy pierwszym dobrym strzale zostają sprawdzone pola nad, pod, na lewo i na prawo od ostatniego trafienia. Jeżeli jest to jedyne dobre pole to bot cofa się do 1 etapu. Jeżeli jednak trafia kolejny raz to w zależności, czy 2 strzał jest horyzontalny czy wertykalny wykonują się kolejne strzały. W przypadku horyzontalnym sprawdzana jest kolejna komórka na prawo od ostatniej trafionej i na lewo od pierwszej albo na odwrót, w zależności od poprzednich ruchów. Jeżeli nic nie zostanie trafione to oznacza że statek o wielkości 2 został ustrzelony. Jeżeli nie to kolejne ruchy są bardzo podobne co do poprzednich. Przed każdym strzałem po 1 trafionym sprawdzane są granice tabeli (czy strzał przypadkiem nie chce za niego wyjść).

```
// //MessageBox.Show("Bot strzela " + AI.Bot_Game_plan.last_good_shot_x.ToString() + " " + AI.Bot_Game_plan.last_good_shot_y.ToString());
if (AI.Bot_Game_plan.total_amount_ships > 0) //sprawdza czy sa statki na planszy
{
    // //MessageBox.Show("sa statki");
    if (AI.Bot_Game_plan.shooted) // sprawdz czy przedtem statek zostal trafiony
    {
        // //MessageBox.Show("HostArray.GetLength(0).ToString()");

        // //MessageBox.Show("poprzednia czesc statku zostala trafiona");
        if (AI.Bot_Game_plan.second_shooted) //sprawdza czy druga czesc statku zostala trafiona
        {
            // //MessageBox.Show("2 shot");
            if (AI.Bot_Game_plan.third_shooted)
            {
                // //MessageBox.Show("4 shot");
                if (AI.Bot_Game_plan.ships_size_four == 0)
                {
                    // //MessageBox.Show("nie ma juz statkow 4 wiec nie ma sensu sprawdzac");
                    AI.Bot_Game_plan.ships_size_three--;
                    sett();
                    goto poczatek;
                }
            }
            if (AI.Bot_Game_plan.right_right_shooted)
            {
                AI.Bot_Game_plan.fourth_good_shot_x = AI.Bot_Game_plan.third_good_shot_x;
                AI.Bot_Game_plan.fourth_good_shot_y = AI.Bot_Game_plan.third_good_shot_y;
                AI.Bot_Game_plan.fourth_good_shot_y_right = AI.Bot_Game_plan.fourth_good_shot_y;
                AI.Bot_Game_plan.fourth_good_shot_y_left = AI.Bot_Game_plan.last_good_shot_y;
                AI.Bot_Game_plan.fourth_good_shot_y_right++;
                AI.Bot_Game_plan.fourth_good_shot_y_left--;
                if (AI.Bot_Game_plan.fourth_good_shot_y_left < 0)
                {
                    if (AI.Bot_Game_plan.fourth_right_right_checked == false && AI.Bot_Game_plan.fourth_good_shot_y_right < 9 && AI.Bot_Game_plan.bot_moves[AI.Bot_Game_plan.fourth_good_shot_x, AI.Bot_Game_plan.fourth_good_shot_y_right] == true)
                    {
                        string szukana = (AI.Bot_Game_plan.fourth_good_shot_x + " " + AI.Bot_Game_plan.fourth_good_shot_y_right).ToString();
                        int index = lista.IndexOf(szukana);
                        // //MessageBox.Show("index: " + index.ToString() + " liczby " + szukana);
                        if (index >= 0) lista.RemoveAt(index);
                    }
                }
            }
        }
    }
}
```


THE SHIP GAME



```
else if (!AI.Bot_Game_plan.fourth_shot_right_right_checked == false && AI.Bot_Game_plan.fourth_shot_y_left == 0 && AI.Bot_Game_plan.bot_moves[AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y_left] == false)
{
    if (!AI.Bot_Game_plan.fourth_shot_right_checked == false && AI.Bot_Game_plan.fourth_shot_y_left == 0)
    {
        string szukana = (AI.Bot_Game_plan.fourth_shot_x + " " + AI.Bot_Game_plan.fourth_shot_y_left).ToString();
        int index = lista.IndexOf(szukana);
        //MessageBox.Show("Szukana: " + szukana);
        if (index == 0) lista.Remove(index);
        AI.Bot_Game_plan.bot_moves[AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y_left] = true;
        AI.Bot_Game_plan.bot_shot_position[AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y_left] = true;
        AI.Bot_Game_plan.total_munitions--;
        AI.Bot_Game_plan.ships_size--;
        AI.Bot_Game_plan.fourth_shot_y = AI.Bot_Game_plan.fourth_shot_y_left;
        //MessageBox.Show("Zostalo: " + AI.Bot_Game_plan.fourth_shot_y);
        AI.Bot_Game_plan.total_munitions++;
        getShipColor(AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y); BackgroundImage = img1;
        sett();
    }
}
else
{
    string szukana = (AI.Bot_Game_plan.fourth_shot_x + " " + AI.Bot_Game_plan.fourth_shot_y_left).ToString();
    int index = lista.IndexOf(szukana);
    //MessageBox.Show("Szukana: " + szukana);
    if (index == 0) lista.Remove(index);
    //MessageBox.Show("Zostalo: " + AI.Bot_Game_plan.fourth_shot_y_left + " " + AI.Bot_Game_plan.fourth_shot_x);
    AI.Bot_Game_plan.bot_moves[AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y_left] = true;
    AI.Bot_Game_plan.fourth_shot_right_right_checked = true;
    getShipColor(AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y); BackgroundImage = img1;
    sett();
}
else
{
    //MessageBox.Show("Zostalo: " + AI.Bot_Game_plan.fourth_shot_y);
    AI.Bot_Game_plan.ships_size--;
    sett();
    goto poczetek;
}
}
else if (!AI.Bot_Game_plan.right_left_shooted)
{
    AI.Bot_Game_plan.fourth_shot_x = AI.Bot_Game_plan.third_shot_x;
    AI.Bot_Game_plan.fourth_shot_y = AI.Bot_Game_plan.third_shot_y;
    AI.Bot_Game_plan.fourth_shot_right_left_checked = true;
    AI.Bot_Game_plan.fourth_shot_y_left = AI.Bot_Game_plan.fourth_shot_y;
    if (!AI.Bot_Game_plan.fourth_shot_right_left_checked == false && AI.Bot_Game_plan.fourth_shot_y_left == 0 && AI.Bot_Game_plan.bot_moves[AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y_left] == false)
    {
        if (!AI.Bot_Game_plan.fourth_shot_x, AI.Bot_Game_plan.fourth_shot_y_left) == true)
    {

```

Gra toczy się do momentu kiedy jeden z graczy nie ustrzeli wszystkich statków. W przypadku bota jest to sprawdzane jako pierwszy warunek. Po ustrzeleniu całego statku wykonuje się metoda sett() która resetuje wszystkie niezbędne zmienne które były wcześniej potrzebne.

```
Odwolania: 28
public void sett()
{
    AI.Bot_Game_plan.shooted = false;
    AI.Bot_Game_plan.second_shooted = false;
    AI.Bot_Game_plan.left_checked = false;
    AI.Bot_Game_plan.right_checked = false;
    AI.Bot_Game_plan.up_checked = false;
    AI.Bot_Game_plan.up_up_checked = false;
    AI.Bot_Game_plan.up_down_checked = false;
    AI.Bot_Game_plan.down_down_checked = false;
    AI.Bot_Game_plan.down_up_checked = false;
    AI.Bot_Game_plan.down_checked = false;
    AI.Bot_Game_plan.left_left_checked = false;
    AI.Bot_Game_plan.right_right_checked = false;
    AI.Bot_Game_plan.left_right_checked = false;
    AI.Bot_Game_plan.right_left_checked = false;
    AI.Bot_Game_plan.right_shooted = false;
    AI.Bot_Game_plan.left_shooted = false;
    AI.Bot_Game_plan.up_shooted = false;
    AI.Bot_Game_plan.down_shooted = false;
    AI.Bot_Game_plan.third_shooted = false;
    AI.Bot_Game_plan.fourth_shot_right_right_right_checked = false;
    AI.Bot_Game_plan.fourth_shot_right_right_left_checked = false;
    AI.Bot_Game_plan.fourth_shot_right_left_left_checked = false;
    AI.Bot_Game_plan.fourth_shot_left_left_left_checked = false;
    AI.Bot_Game_plan.fourth_shot_left_left_right_checked = false;
    AI.Bot_Game_plan.fourth_shot_left_right_right_checked = false;
    AI.Bot_Game_plan.fourth_shot_up_up_up_checked = false;
    AI.Bot_Game_plan.fourth_shot_up_up_down_checked = false;
    AI.Bot_Game_plan.fourth_shot_up_down_down_checked = false;
    AI.Bot_Game_plan.fourth_shot_down_down_down_checked = false;
    AI.Bot_Game_plan.fourth_shot_down_down_up_checked = false;
    AI.Bot_Game_plan.fourth_shot_down_down_up_checked = false;
}
```

Podczas gry zapisują się podstawowe statystyki, takie jak zniszczone elementy statków, wygrane potyczki oraz rozegrane gry. Wszystko zapisuje aktualizuje się w momencie ukończenia gry i zostaje zapisane do kontrolki SCOREBOARD. Wszystkie zmienne są przechowywane klasie "AI"

THE SHIP GAME



```
void whenGameEndsSuccessfully(bool res)
{
    czasGry.Stop();
    int wins = int.Parse(File.ReadAllText("wins"));
    int PlayedBattles = int.Parse(File.ReadAllText("PlayedBattles"));
    int DestroyedShips = int.Parse(File.ReadAllText("DestroyedShips"));
    int TimeInGame = int.Parse(File.ReadAllText("TimeInGame"));
    File.WriteAllText("wins", (wins + (res ? 1 : 0)).ToString());
    File.WriteAllText("PlayedBattles", (PlayedBattles + 1).ToString());
    File.WriteAllText("DestroyedShips", (DestroyedShips + shipsShot).ToString());
    int czas = int.Parse(czasGry.ElapsedMilliseconds.ToString()) / 6000;
    File.WriteAllText("TimeInGame", (TimeInGame + czas).ToString());
    FormMain.MainGlobals.wins = (wins + 1).ToString();
    FormMain.MainGlobals.PlayedBattles = (PlayedBattles + 1).ToString();
    FormMain.MainGlobals.DestroyedShips = (DestroyedShips + shipsShot).ToString();
    FormMain.MainGlobals.TimeInGame = (TimeInGame + czas).ToString();
}
```

```
public static class Bot_Game_plan
{
    public static int total_amount_ships = 22;
    public static int ships_destroyed = 0;
    public static int[,] ships_destroyed_position = new int[10, 10];
    public static bool[,] bot_ships_position = new bool[10, 10];
    public static int ships_size_one = 4;
    public static int ships_size_two = 3;
    public static int ships_size_three = 2;
    public static int ships_size_four = 1;
    public static bool up_checked = false;
    public static bool right_checked = false;
    public static bool right_right_checked = false;
    public static bool right_left_checked = false;
    public static bool left_left_checked = false;
    public static bool left_right_checked = false;
    public static bool up_up_checked = false;
    public static bool up_down_checked = false;
    public static bool down_checked = false;
    public static bool down_down_checked = false;
    public static bool down_up_checked = false;
    public static bool left_checked = false;
    public static bool[,] bot_moves = new bool[100, 100];
    public static bool[,] bot_good_shots_position = new bool[100, 100];
    public static int amount_bot_good_shots = 0;
    public static int total_amount_shots = 0;
    public static bool shooted = false;
    public static bool second_shooted = false;
    public static bool third_shooted = false;
    public static bool fourth_shooted = false;
    public static int last_good_shot_x = 0;
    public static int last_good_shot_y = 0;
    public static int second_good_shot_y_right = 0;
    public static int second_good_shot_y_left = 0;
    public static int second_good_shot_y = 0;
    public static int second_good_shot_x = 0;
    public static int second_good_shot_x_down = 0;
    public static int second_good_shot_x_up = 0;
    public static int third_good_shot_x = 0;
    public static int third_good_shot_y = 0;
    public static int third_good_shot_x_down = 0;
    public static int third_good_shot_x_up = 0;
    public static int third_good_shot_y_right = 0;
    public static bool right_right_shooted = false;
    public static bool right_left_shooted = false;
    public static bool left_right_shooted = false;
}
```

SCOREBOARD			
WINS	?	3	?
PLAYED BATTLES	?	11	?
DESTROYED PIECES	?	80	?
TIME IN GAME	?	1 MINS	?
BACK			

W momencie ukończenia rozgrywki pojawia się opcja rozpoczęcia gry na nowo, która przekierowuje nas do ponownego rozstawienia statków, lub opuszczenia rozgrywki która przerzuci nas do okienka z trybami gry. Wywołują to metody IWon() lub BotWon() w zależności od tego kto wygra

THE SHIP GAME



```

C#
private void IWon()//host action
{
    ((FormMain)Parent).MSG("You won");
    GameNotify.Text = "you won";
    this.Invoke(new Action() => { tableLayoutPanel20.Visible = true; }));
}

1 odwołanie
private void BotWon()
{
    ((FormMain)Parent).MSG("Bot won");
    GameNotify.Text = "Bot won";
    this.Invoke(new Action() => { tableLayoutPanel20.Visible = true; }));
}
    
```

me: username connected											GAME		opponent: ai connected										
	a	b	c	d	e	f	g	h	i	j	it is opponent turn		a	b	c	d	e	f	g	h	i	j	
1												1											
2												2											
3												3											
4												4											
5												5											
6												6											
7												7											
8												8											
9												9											
10											10												
DO YOU WANT PLAY AGAIN?											exit	 x-1+1											
YES												 x-1+1											
NO												 x-1+1											
												 x-1+1											

THE SHIP GAME



2) Projekt systemu

Projekt używa języka C# w połączeniu z platformą .NET Framework 4.8.

Każdy oddzielny widok w aplikacji jest zaimplementowany jako Kontrolka Użytkownika Windows Forms.

Wykorzystane biblioteki:

- Biblioteki zawarte w standardzie .NET Framework 4.8
- NAudio (<https://github.com/naudio/NAudio>)

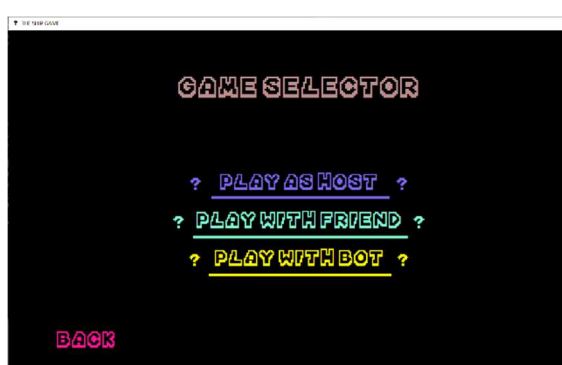
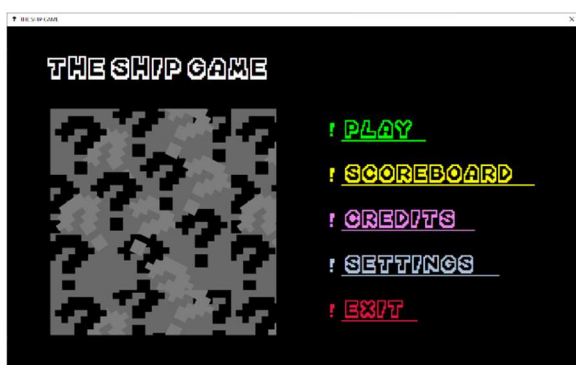
Komunikacja między graczami za pomocą protokołu WebSocket.

Pliki zasobów przechowywane w systemie plików i używane w aplikacji poprzez dostęp w czasie pracy (pliki nie są zawarte w pamięci statycznej pliku wykonywalnego).

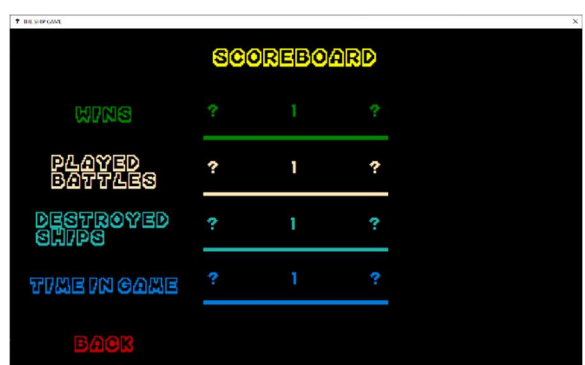
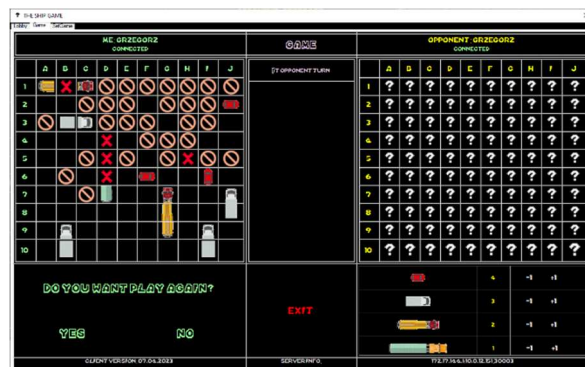
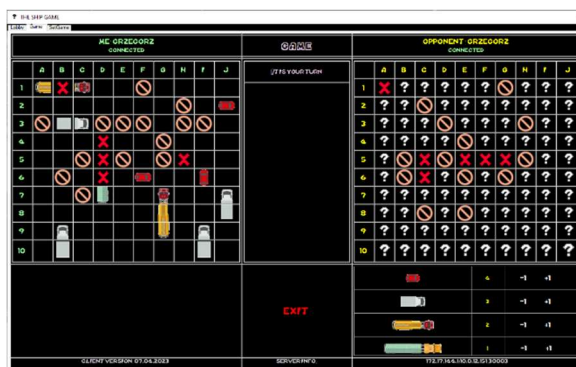
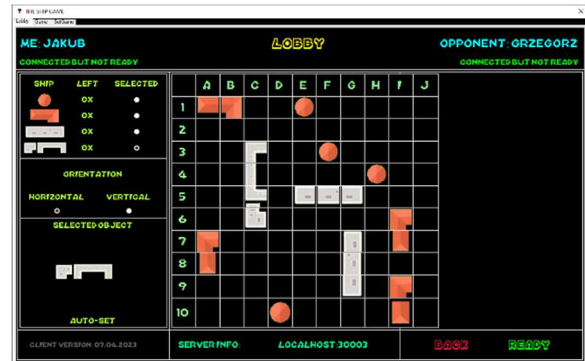
3) Realizacja systemu – interfejs graficzny

Dzieli się na:

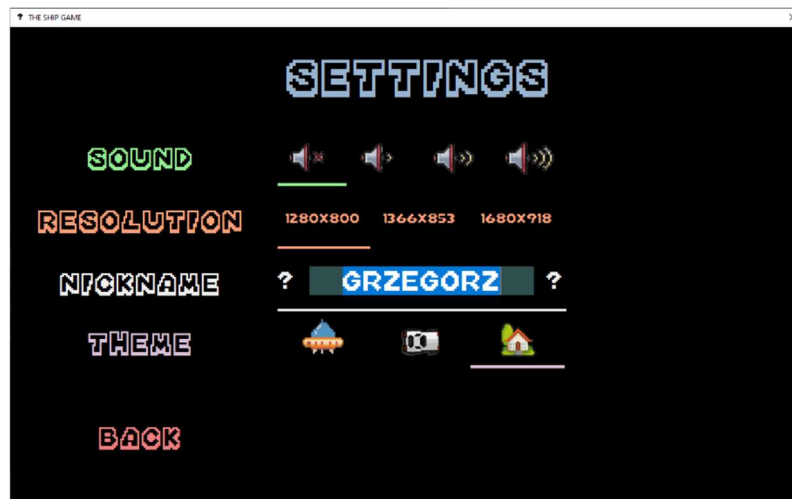
- Menu
- Wybór trybu (gra jako host, gra z innym użytkownikiem, gra z botem)
- Lobby
- Widok rozgrywki
- Opcje
- Statystyki



THE SHIP GAME



THE SHIP GAME



4) Protokół rozbieżności

Udało się zaimplementować:

- Gra z innym członkiem sieci LAN
- Gra z botem
- Zapisywanie statystyk (ilość wygranych gier, ilość zagranych gier, zniszczone statki)
- Różne motywy statków
- Ponowna gra z tym samym przeciwnikiem bez przymusu ponownego łączenia się
- Automatyczne ustawianie statków na prośbę gracza
- Konfigurowalne nazwy graczy

Nie udało się zaimplementować:

- Zapętlenie muzyki w tle
- Czas rozgrywki (Pozbawiony sensu z powodu, że rozgrywka trwa około 2-3 minut)

Wszystkie główne założenia projektu udało się wdrożyć w zadanym czasie.

Autorzy: Grzegorz Lademann, Jakub Lidzbarski, Cezary Piernikowski